VELLORE INSTITUTE OF TECHNOLOGY (VIT) VELLORE

BACSE203 P -- Computer Networks Lab (Innovative Lab Assessment (ELA 1.0))

PROJECT TITLE: NetMirage: Autonomous Cyber-Range with Zero-Egress Sandboxing & Adaptive Deception Routing

FACULTY GUIDE: Dr. Sudha M, Professor, SCOPE | **SCHOOL**: School of Computer Science and Engineering

CANDIDATES: Abhisumat Panda (25BYB0202) | Atharva Parmar (25BYB0180) | Manas Patel (25BYB0190) | Varun Agarwal (25BYB0186)

SECTION 1: SYLLABUS-MAPPED NETWORKING QUESTIONS (38 QUESTIONS)

TOPIC FOCUS: MODULE 1: NETWORKING FOUNDATIONS

Q1: OSI vs. TCP/IP Model Mapping in NetMirage

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

The OSI and TCP/IP models organize network communication into layered steps so hardware, routing, and applications can talk without interfering. In NetMirage, every simulated attack and defense action maps directly from Layer 2 Docker virtual bridges up to Layer 7 WebSocket telemetry streams.

DETAILED TECHNICAL BREAKDOWN:

The 7-layer OSI model (Physical, Data Link, Network, Transport, Session, Presentation, Application) maps onto the 4-layer TCP/IP stack (Link, Internet, Transport, Application). In NetMirage: Layer 2 uses Linux virtual ethernet (veth) pairs attached to Docker bridge networks with isolated MAC addresses; Layer 3 assigns private IPv4 addresses across 5 /24 subnets with zero-egress routing; Layer 4 handles TCP segment creation (SYN flags, sequence numbers) and UDP datagrams; Layer 7 hosts application daemons (OpenSSH on port 22, HTTP/1.1 on port 80, BIND 9 on port 53, Postfix on port 25) serialized via REST JSON and full-duplex WebSockets.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: `app_build/docker/docker-compose.lab.yml (L3-48 for L2/L3 bridges), app_build/backend/intel/analyzer.py (ProtocolAnalyzer.analyze_action_event, L27-128 for L4/L7 protocol inspection), and app_build/backend/routes.py (websocket_events, L264-300 for L7 streaming).`

TOPIC FOCUS: MODULE 1: NETWORKING FOUNDATIONS

Q2: Protocol Encapsulation & De-multiplexing Across Layers

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Encapsulation is like putting a letter inside nested envelopes, where each layer adds its own header before sending data down the wire. When a packet arrives, each layer strips off its corresponding envelope header to deliver the payload to the intended application.

DETAILED TECHNICAL BREAKDOWN:

At the sender, application payload data is prepended with a TCP header (specifying source/destination ports, sequence numbers), creating a Transport Layer segment. This segment is encapsulated in an IPv4 datagram by adding an IP header (20 bytes minimum, protocol field = 0x06 for TCP, TTL, source/destination IPs). At the Data Link layer, an Ethernet II frame header (14 bytes: 6-byte dest MAC, 6-byte src MAC, 2-byte EtherType = 0x0800 for IPv4) and 4-byte Frame Check Sequence (FCS) are added. At the receiver, de-multiplexing uses EtherType to route to IPv4, the IP Protocol field to route to TCP, and the destination TCP port to pass data to the target socket.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: `app_build/backend/intel/analyzer.py (ProtocolAnalyzer.analyze_scapy_packet, L130-153) uses Scapy to decapsulate raw frame bytes (Ether -> IP -> TCP/UDP/ICMP) and extract header fields.`

TOPIC FOCUS: MODULE 1: NETWORKING FOUNDATIONS

Q3: Network Sockets API & Transport Layer Abstraction

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

A network socket is a software doorway that lets programs send and receive data across a network using an IP address and port number. NetMirage uses Python asynchronous sockets to run controlled network handshakes without needing operating system shell commands.

DETAILED TECHNICAL BREAKDOWN:

The Berkeley Sockets API abstracts Layer 4 transport communication into file descriptor-like endpoints defined by the 5-tuple: {Source IP, Source Port, Destination IP, Destination Port, Transport Protocol}. Stream sockets (SOCK_STREAM) provide reliable, sequenced byte streams using TCP, while datagram sockets (SOCK_DGRAM) provide connectionless, unreliable transmission using UDP. In NetMirage, connect_tcp uses Python's asynchronous network stack (asyncio.open_connection or low-level socket.socket(socket.AF_INET, socket.SOCK_STREAM)) to issue TCP handshakes and measure round-trip establishment latency without spawning external binaries like nc or nmap.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/core/action_broker.py (ActionBroker._exec_connect_tcp, L119-124 & execute_action, L24-59).

TOPIC FOCUS: MODULE 1: NETWORKING FOUNDATIONS

Q4: Network Performance Metrics: Throughput, Bandwidth, Delay, and Latency

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Bandwidth is the maximum width of the highway, while throughput is how many cars actually pass each second. Latency is the total time it takes for a single packet to travel from the sender to the receiver and back.

DETAILED TECHNICAL BREAKDOWN:

Total network delay is governed by four constituent components: D_total = D_proc + D_queue + D_trans + D_prop, where nodal processing delay D_proc is time spent parsing headers and checksums; queuing delay D_queue is buffer wait time; transmission delay D_trans = L/R (packet length L in bits divided by link rate R in bps); and propagation delay D_prop = d/s (physical distance d divided by signal speed s approx 2x10^8 m/s). Round-Trip Time (RTT) measures the elapsed time from packet transmission to receipt of its acknowledgment.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/core/action_broker.py (execute_action, L24-59) uses time.perf_counter() to benchmark high-precision elapsed duration in milliseconds (latency_ms), recorded in models.AgentAction.latency_ms.

TOPIC FOCUS: MODULE 2: DIRECT LINKS

Q5: Framing Protocols: Byte-Oriented (PPP) vs. Bit-Oriented (HDLC)

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Framing breaks a continuous stream of bits into distinct packages with clear start and end markers so the receiver knows where each message begins. Byte-oriented protocols like PPP use special flag characters and byte-stuffing, while bit-oriented protocols like HDLC use 6-bit flag patterns and zero-bit insertion.

DETAILED TECHNICAL BREAKDOWN:

In Byte-Oriented framing (such as Point-to-Point Protocol; PPP / RFC 1661), frames begin and end with an 8-bit flag byte (0x7E / 01111110). If the flag byte or escape byte (0x7D) appears inside the payload, the sender inserts an escape byte (Byte Stuffing). In Bit-Oriented framing (such as High-Level Data Link Control; HDLC / ISO 13239), the flag sequence is exactly 01111110 (six consecutive 1s). Whenever the sender encounters five consecutive 1s in the data stream, it automatically stuffs a '0' bit (Bit Stuffing); the receiver reverses this by dropping any 0 that immediately follows five consecutive 1s, preserving frame synchronization.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: Syllabus Module 2 (Framing: Byte-Oriented Protocols PPP, Bit-Oriented Protocols HDLC); foundational link-layer concept.

TOPIC FOCUS: MODULE 2: DIRECT LINKS

Q6: Ethernet Framing (IEEE 802.3), MAC Addressing & MTU

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Ethernet frames wrap data in digital envelopes marked with hardware MAC addresses so network switches know which physical device receives each packet. The Maximum Transmission Unit (MTU) defines the largest data payload (normally 1500 bytes) that can travel inside a single frame without being chopped up.

DETAILED TECHNICAL BREAKDOWN:

Standard Ethernet II frames consist of a 7-byte Preamble (alternating 1s and 0s for clock synchronization), a 1-byte Start Frame Delimiter (SFD: 10101011), a 6-byte Destination MAC, a 6-byte Source MAC, a 2-byte EtherType (0x0800 for IPv4, 0x0806 for ARP), a Payload of 46 to 1500 bytes (enforcing the default MTU of 1500 bytes), and a 4-byte Frame Check Sequence (CRC-32). If an IP packet exceeds the interface MTU (1500 bytes), the IPv4 layer must perform fragmentation using the Identification, Flags (DF/MF), and Fragment Offset header fields.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: `app_build/backend/intel/analyzer.py (analyze_scapy_packet, L130-153) inspects the Ethernet layer via scapy.all.Ether, validating hardware MACs and frame byte lengths.`

TOPIC FOCUS: MODULE 2: DIRECT LINKS

Q7: Error Detection: Internet Checksum Algorithm

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

The Internet Checksum adds up all the 16-bit words in a packet header and inverts the result so the receiver can detect if any bits flipped during transit. If the receiver adds all words together including the checksum and doesn't get all 1s, it knows the packet was corrupted and throws it away.

DETAILED TECHNICAL BREAKDOWN:

Defined in RFC 1071, the Internet Checksum treats data as a sequence of 16-bit integers. The sender sums these integers using 1's complement arithmetic (where carry bits past the 16th bit are wrapped around and added back into the least significant bit). The final 16-bit sum is bitwise inverted (1's complement) and stored in the header checksum field. Upon receipt, the receiver calculates the 1's complement sum over all 16-bit words including the transmitted checksum; if transmission was error-free, the resulting sum must evaluate to 0xFFFF (all 1s in 1's complement arithmetic, or negative zero).

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: `Syllabus Module 2 & Indicative Experiment 2. Evaluated in NetMirage protocol inspection via Scapy packet validation in app_build/backend/intel/analyzer.py.`

TOPIC FOCUS: MODULE 2: DIRECT LINKS

Q8: Error Detection: Cyclic Redundancy Check (CRC-32) & FCS

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

CRC-32 treats the entire packet data as a huge binary polynomial and divides it by a standard mathematical divisor to generate a 32-bit remainder. The receiver performs the same division; if the remainder isn't zero, it detects burst transmission errors with near-perfect reliability.

DETAILED TECHNICAL BREAKDOWN:

Cyclic Redundancy Check (CRC) is an algebraic polynomial code based on Galois Field modulo-2 arithmetic (where addition and subtraction are equivalent to bitwise XOR, without carries or borrows). Given a message bitstring M(x) of length k and a predefined generator polynomial G(x) of degree r (such as the standard Ethernet CRC-32 polynomial G(x) = x^32 + x^26 + x^23 + x^22 + x^16 + x^12 + x^11 + x^10 + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1), the sender computes the remainder R(x) = [M(x) * x^r] mod G(x) and appends R(x) as the 4-byte Frame Check Sequence (FCS). The receiver divides [M(x) * x^r + R(x)] by G(x); any non-zero remainder signals corrupted bits.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: `Syllabus Module 2 & Lab Exp 2; simulated in Ethernet frame integrity validation within Scapy parsing in app_build/backend/intel/analyzer.py.`

TOPIC FOCUS: MODULE 2: DIRECT LINKS

Q9: Error Correction: Hamming Code & Syndrome Calculation

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Hamming code embeds calculated parity bits at power-of-two positions in data so the receiver cannot only tell if an error occurred, but pinpoint the exact flipped bit and fix it automatically. This gives single-bit error correction without needing to retransmit the entire packet.

DETAILED TECHNICAL BREAKDOWN:

In an (n, k) Hamming code (such as Hamming(7, 4)), m data bits are augmented with r parity bits such that 2^r >= m + r + 1, where n = m + r. Parity bits are placed at bit positions that are powers of 2 (1, 2, 4, 8, ...), with each parity bit covering data bit positions whose binary representation has a 1 in that parity bit's corresponding power-of-two position. At the receiver, parity recalculation generates an r-bit binary integer known as the 'syndrome.' If the syndrome is zero, no single-bit error occurred; if non-zero, the syndrome directly indicates the 1-indexed bit position of the inverted bit, enabling instantaneous hardware bit-flip correction.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: Syllabus Module 2 & Lab Exp 2. Core theoretical concept tested in Computer Networks viva examinations.

TOPIC FOCUS: MODULE 2: DIRECT LINKS

Q10: Data Link Flow Control: Stop-and-Wait vs. Sliding Window

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Stop-and-Wait forces the sender to wait for an acknowledgment after every single packet before sending the next one, which wastes channel capacity on long links. Sliding Window allows the sender to transmit multiple packets up to a set window limit before waiting, keeping the network pipeline full.

DETAILED TECHNICAL BREAKDOWN:

In Stop-and-Wait, link utilization is U = T_trans / (T_trans + 2 * T_prop), which approaches zero on links with a large Bandwidth-Delay Product (BDP). Sliding Window protocols maintain sequence numbers and a transmission window W_s. In Go-Back-N, the receiver only accepts frames in strict sequence (discarding out-of-order frames) and sends cumulative ACKs; on timeout, the sender retransmits all unacknowledged frames starting from the oldest outstanding packet. In Selective Repeat, the receiver buffers out-of-order frames within a receive window W_r <= 2^(n-1) and sends individual selective ACKs, so the sender retransmits only the specific corrupted or dropped frame.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: Syllabus Module 2 & Lab Exp 3. Mapped in Layer 4 TCP connection state and sliding window emulation in app_build/backend/core/action_broker.py and app_build/backend/intel/analyzer.py.

TOPIC FOCUS: MODULE 2: DIRECT LINKS

Q11: Multiple Access Protocols: CSMA/CD vs. CSMA/CA

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

CSMA/CD tells devices on wired Ethernet to listen before transmitting and immediately stop if they hear a collision with another transmission. Wireless networks (Wi-Fi) cannot detect collisions while transmitting, so they use CSMA/CA to avoid collisions by waiting random backoff intervals and exchanging RTS/CTS reservation messages.

DETAILED TECHNICAL BREAKDOWN:

Carrier Sense Multiple Access with Collision Detection (CSMA/CD; IEEE 802.3) requires that minimum frame transmission time T_trans >= 2 * T_prop, ensuring that a transmitting station detects collisions before finishing its frame transmission. Upon collision, stations abort transmission, transmit a 32-bit jam signal, and invoke the Truncated Binary Exponential Backoff algorithm (waiting k * 512 bit-times, where k in [0, 2^i - 1] for collision iteration i <= 10). In contrast, wireless interfaces (IEEE 802.11) suffer from the 'hidden terminal' problem and high dynamic range, preventing simultaneous transmission and listening; they utilize Collision Avoidance (CSMA/CA) with interframe spaces (DIFS/SIFS), random backoff timers, and optional Request-to-Send / Clear-to-Send (RTS/CTS) virtual carrier sensing.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: Syllabus Module 2. Mirrored in NetMirage by Docker bridge switching where collision domains are segmented into isolated point-to-point virtual links.

TOPIC FOCUS: MODULE 3: INTERNETWORKING

Q12: Virtual LANs (VLANs) & Docker Bridge Network Virtualization

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

VLANs slice a single physical network into isolated virtual compartments so computers in one department cannot eavesdrop on or access computers in another. NetMirage uses 5 independent Docker bridge networks to isolate the edge gateway, honeypots, corporate services, and monitoring tap.

DETAILED TECHNICAL BREAKDOWN:

VLANs (IEEE 802.1Q) introduce a 4-byte tag into Ethernet headers containing a 12-bit VLAN Identifier (VID), allowing up to 4094 isolated broadcast domains over shared physical switches. In Linux container environments, Docker simulates VLAN segmentation using kernel bridge interfaces (br-xxx). Each container is attached via a Virtual Ethernet (veth) pair—one interface inside the container network namespace (eth0) and its peer attached to the Linux bridge. Broadcast traffic (such as ARP requests and DHCP discovers) is confined strictly to the container's attached bridge, preventing lateral layer-2 leakage between lab subnets.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/docker/docker-compose.lab.yml (L3-48 defines edge_net, ssh_net, web_net, services_net, monitor_net with internal: true).

TOPIC FOCUS: MODULE 3: INTERNETWORKING

Q13: IPv4 Subnetting Math for All 5 NetMirage /24 Segments

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Every NetMirage subnet uses a /24 prefix, which reserves 24 bits for the network and leaves 8 bits for host addresses. This creates exactly 256 IP addresses per segment, with 254 usable by active hosts after reserving the network and broadcast addresses.

DETAILED TECHNICAL BREAKDOWN:

A /24 subnet mask corresponds to binary 11111111.11111111.11111111.00000000 (decimal 255.255.255.0). The host portion has h = 32 - 24 = 8 bits, yielding 2^8 = 256 total addresses. The number of usable host addresses is 2^h - 2 = 254, because the all-zeros host address represents the Subnet/Network ID and the all-ones host address represents the Directed Broadcast Address. NetMirage segment calculations:

- edge (10.0.10.0/24): Subnet ID 10.0.10.0, Gateway 10.0.10.1, Usable Range 10.0.10.1 - 10.0.10.254, Broadcast 10.0.10.255. Hosts: Gateway (.42), Adversary (.15).
- ssh (10.0.20.0/24): Subnet ID 10.0.20.0, Gateway 10.0.20.1, Usable Range 10.0.20.1 - 10.0.20.254, Broadcast 10.0.20.255. Host: Cowrie SSH Lure (.10).
- web (10.0.30.0/24): Subnet ID 10.0.30.0, Gateway 10.0.30.1, Usable Range 10.0.30.1 - 10.0.30.254, Broadcast 10.0.30.255. Host: Decoy Web Portal (.10).
- services (10.0.40.0/24): Subnet ID 10.0.40.0, Gateway 10.0.40.1, Usable Range 10.0.40.1 - 10.0.40.254, Broadcast 10.0.40.255. Hosts: DNS Lure (.10), SMTP Lure (.20).
- monitor (10.0.50.0/24): Subnet ID 10.0.50.0, Gateway 10.0.50.1, Usable Range 10.0.50.1 - 10.0.50.254, Broadcast 10.0.50.255. Hosts: Telemetry Collector (.10), Quarantine Sinkhole (.99).

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/docker/docker-compose.lab.yml (L3-48), app_build/backend/config.py (ALLOWED_LAB_CIDRS), and app_build/backend/routes.py (get_topology, L27-73).

TOPIC FOCUS: MODULE 3: INTERNETWORKING

Q14: Classful vs. Classless Addressing (CIDR) & Longest Prefix Match

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Classful addressing divided the world into rigid A, B, and C blocks that wasted millions of addresses, while CIDR lets engineers carve networks into any size using a slash prefix. Routers use Longest Prefix Match to pick the most specific route in their table when forwarding packets.

DETAILED TECHNICAL BREAKDOWN:

Classful IPv4 assigned fixed octet boundaries: Class A (/8, leading bit 0, 16.7M hosts), Class B (/16, leading bits 10, 65,534 hosts), Class C (/24, leading bits 110, 254 hosts), Class D (Multicast: 1110), and Class E (Experimental: 1111). RFC 1519 introduced Classless Inter-Domain Routing (CIDR), eliminating fixed boundaries and allowing arbitrary prefix masks (/n). When forwarding an IP packet, routers execute the Longest Prefix Match algorithm: they perform bitwise AND operations between the packet's destination IP and each routing table entry's subnet mask; if multiple routes match, the router selects the entry with the largest prefix length n, representing the most specific forwarding route.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/core/safety_policy.py (SafetyPolicyEngine.is_ip_in_lab, L44-49) parses CIDRs using Python's ipaddress.ip_network and performs prefix containment matching against self.allowed_networks.

TOPIC FOCUS: MODULE 3: INTERNETWORKING

Q15: Datagram Forwarding, Routing Table Lookup & Network as a Graph

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

A network is a graph where computers and routers are nodes and cables are edges with cost metrics. When a packet arrives, the router checks its routing table to decide which connected neighbor should receive the packet next to reach its destination.

DETAILED TECHNICAL BREAKDOWN:

In the Internet datagram model, each IP packet is forwarded independently across hops without prior path reservation. Network topology is formalized as a weighted graph G = (V, E), where vertices V represent routers/hosts and edges E represent links with assigned metric costs (hop count, delay, bandwidth, or administrative cost). Routers maintain a Forwarding Information Base (FIB) derived from routing protocol graph computations. Upon ingress, the router decrements the IPv4 header Time-to-Live (TTL) field by 1 (dropping the packet and returning an ICMP Time Exceeded message if TTL reaches 0), recalculates the header checksum, and forwards the packet out the next-hop egress interface.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/routes.py (get_topology, L27-116) represents NetMirage topology as graph segments, nodes, and highlighted paths, rendered visually on the React canvas in app_build/frontend/src/components/NetworkTopology.tsx.

TOPIC FOCUS: MODULE 3: INTERNETWORKING

Q16: Address Resolution Protocol (ARP) & ARP Cache Poisoning (Spoofing) Detection

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

ARP translates a known Layer-3 IP address into a physical Layer-2 MAC address by broadcasting a question across the local network. ARP spoofing happens when an attacker sends fake ARP responses to trick computers into routing traffic through the attacker's machine.

DETAILED TECHNICAL BREAKDOWN:

When host A needs to deliver an IP datagram to host B on the same local subnet, it checks its local ARP cache table. If no entry exists, A broadcasts an ARP Request frame (Ethernet Destination: FF:FF:FF:FF:FF:FF, ARP Opcode: 1) asking 'Who has IP X? Tell IP Y.' Host B matches IP X and unicasts an ARP Reply (Opcode: 2) containing its MAC address. Because standard ARP is stateless and unauthenticated, an adversary can broadcast gratuitous ARP replies mapping legitimate IPs (such as the gateway) to the attacker's MAC address (ARP Poisoning / MITRE T1557). NetMirage's passive collector monitors ARP frames and flags sudden MAC-to-IP binding mutations as spoofing candidates.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/intel/analyzer.py (L130-153) and app_build/backend/intel/mitre_mapper.py (arp_spoof_candidate mapped to MITRE T1557 Adversary-in-the-Middle with 0.92 confidence, L56-61).

TOPIC FOCUS: MODULE 3: INTERNETWORKING

Q17: Dynamic Host Configuration Protocol (DHCP) & Broadcast Domain Isolation

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

DHCP automatically assigns IP addresses, subnet masks, and default gateways to devices when they join a network. It uses a 4-step conversation called DORA (Discover, Offer, Request, Acknowledge) over UDP ports 67 and 68.

DETAILED TECHNICAL BREAKDOWN:

DHCP operates over UDP via the four-stage DORA exchange:

1. DHCP Discover: Client broadcasts a UDP datagram (0.0.0.0:68 -> 255.255.255.255:67) requesting an IP.
2. DHCP Offer: Server unicasts or broadcasts an offer (Server_IP:67 -> Offered_IP:68 or broadcast) specifying an available IP, subnet mask, lease duration, and gateway.
3. DHCP Request: Client broadcasts its acceptance of the offered lease, signaling rejection to any other competing DHCP servers.
4. DHCP ACK: Server acknowledges the binding, finalizing the client configuration.

Because DHCP Discovers are Layer-2 broadcasts (FF:FF:FF:FF:FF:FF), they cannot cross router interfaces or Docker bridge boundaries. In NetMirage, each /24 lab subnet is an isolated broadcast domain, preventing rogue DHCP starvation attacks from propagating outside their assigned bridge.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/docker/docker-compose.lab.yml (L3-48 isolates broadcast domains for each subnet).

TOPIC FOCUS: MODULE 3: INTERNETWORKING

Q18: Internet Control Message Protocol (ICMP) & Echo Probes (send_icmp_probe)

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

ICMP is the diagnostic messaging protocol used by ping and traceroute to verify if remote computers are online and report delivery errors. NetMirage uses ICMP echo requests to verify network reachability across lab segments and measures round-trip time.

DETAILED TECHNICAL BREAKDOWN:

Encapsulated directly within IPv4 datagrams (IP Protocol number 0x01), ICMP provides network-layer diagnostics and error reporting. The sender dispatches an ICMP Echo Request (Type 8, Code 0) with a 16-bit Identifier, Sequence Number, and payload. A responsive target returns an ICMP Echo Reply (Type 0, Code 0) echoing the identical identifier and payload. ICMP error messages (such as Destination Unreachable: Type 3, or Time Exceeded: Type 11) include the IPv4 header plus first 8 bytes of the offending datagram to enable transport-layer correlation.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: `app_build/backend/core/action_broker.py (ActionBroker._exec_send_icmp_probe, L164-176) executes ICMP probes, and app_build/backend/intel/analyzer.py (L112-127) extracts payload size, TTL, and sequence numbers.`

TOPIC FOCUS: MODULE 3: INTERNETWORKING

Q19: Zero-Egress Network Architecture (internal: true) & Default Gateway Omission

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Zero-egress means network traffic can only move between lab containers and is completely blocked from reaching the host machine or the public internet. This guarantees that simulated attacks can never leak malware or scan external networks by mistake.

DETAILED TECHNICAL BREAKDOWN:

In Docker Compose, applying internal: true to a bridge network instructs the Docker daemon to configure the Linux software bridge without an outbound default route (0.0.0.0/0). Linux kernel iptables rules are automatically written into the DOCKER-ISOLATION and DOCKER filter chains:

-A FORWARD -i br-xxx ! -o br-xxx -j DROP

Any packet originating from a container interface whose destination IP is outside the bridge subnet is dropped by the kernel netfilter before traversing physical network cards or host loopback interfaces (127.0.0.1), guaranteeing mathematical egress isolation.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: `app_build/docker/docker-compose.lab.yml (L6, L15, L24, L33, L42: internal: true on all 5 networks) and docs/SECURITY_BOUNDARIES.md (L33-54).`

TOPIC FOCUS: MODULE 3: INTERNETWORKING

Q20: Network Address Translation (NAT): SNAT vs. DNAT vs. PAT

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

NAT changes IP addresses inside packet headers as they travel through a router so multiple private computers can share one public IP address. In NetMirage, we deliberately disable NAT to keep direct, uncorrupted IP visibility across all lab segments for authentic intrusion detection.

DETAILED TECHNICAL BREAKDOWN:

Source NAT (SNAT) alters the source IP of outbound packets to a public routable IP; Destination NAT (DNAT) modifies the destination IP of inbound packets (port forwarding); and Port Address Translation (PAT / NAPT) multiplexes multiple private IP sockets onto a single public IP by tracking distinct source transport port numbers in a kernel connection tracking (conntrack) table. In standard Docker, nat iptables masquerading rewrites container source IPs to the host IP. In NetMirage, zero-egress bridges operate in routed private IP mode without masquerade NAT, preserving authentic adversary IP identities (10.0.10.15) throughout telemetry and honeypot inspection.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: `app_build/docker/docker-compose.lab.yml and docs/SECURITY_BOUNDARIES.md.`

TOPIC FOCUS: MODULE 3: INTERNETWORKING

Q21: Routing Protocols Theory: Distance-Vector (RIP) vs. Link-State (OSPF)

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Distance-Vector routing (RIP) works like road signs where each router tells its neighbors how far destinations are, but it suffers from slow updates and counting loops. Link-State routing (OSPF) lets every router build a complete map of the whole network and run Dijkstra's algorithm to find the absolute shortest path.

DETAILED TECHNICAL BREAKDOWN:

Routing Information Protocol (RIP; RFC 2453) is a Distance-Vector protocol based on the Distributed Bellman-Ford algorithm: D_x(y) = min_v { c(x, v) + D_v(y) }. Routers broadcast periodic vector updates every 30 seconds; RIP uses hop count as metric (max 15 hops; 16 = infinity) and relies on Split Horizon and Poison Reverse to mitigate the count-to-infinity problem. In contrast, Open Shortest Path First (OSPF; RFC 2328) is an interior Link-State protocol using Dijkstra's Shortest Path First algorithm. Routers flood Link-State Advertisements (LSAs) via Reliable Flooding to populate an identical Link-State Database (LSDB), representing the entire network topology graph, and compute loop-free shortest path trees rooted at each node.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: Syllabus Module 3 & Indicative Exp 5; conceptual foundation for NetMirage's dynamic deception route evaluation in app_build/backend/intel/deception_policy.py.

TOPIC FOCUS: MODULE 3: INTERNETWORKING

Q22: Interdomain Routing & Autonomous Systems (BGP / Path Vector)

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

BGP is the postal system of the global internet, connecting huge independent networks called Autonomous Systems. Instead of looking at individual hops, BGP passes entire path lists to prevent routing loops between countries and internet service providers.

DETAILED TECHNICAL BREAKDOWN:

Border Gateway Protocol (BGP-4; RFC 4271) is the de facto inter-Autonomous System (AS) routing protocol. Operating over reliable TCP port 179, BGP is a Path-Vector protocol that advertises network reachability via Network Layer Reachability Information (NLRI) coupled with path attributes. Key attributes include AS_PATH (a sequence of AS numbers traversed, providing instant loop prevention: an AS immediately drops any route advertisement containing its own AS number), NEXT_HOP (the IP address of the border router), and LOCAL_PREF (administrative egress preference). BGP enables policy-based routing rather than strict shortest-path optimization.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: Syllabus Module 3 (Global Internet - Routing Areas, Interdomain Routing - BGP).

TOPIC FOCUS: MODULE 3: INTERNETWORKING

Q23: IPv4 vs. IPv6 Architecture & Packet Header Differences

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

IPv4 uses 32-bit addresses which gives 4.3 billion addresses and has run out, while IPv6 uses 128-bit addresses providing practically limitless addresses. IPv6 also streamlined its header by removing checksums and fragmentation fields to help routers process packets faster.

DETAILED TECHNICAL BREAKDOWN:

IPv4 headers have variable length (20 to 60 bytes with options), a 16-bit Internet Checksum recalculated at every hop, and 8 fields devoted to in-path fragmentation (Identification, Flags, Fragment Offset). IPv6 features a fixed 40-byte base header (improving hardware pipelining), eliminates intermediate router fragmentation (forcing Path MTU Discovery and source-only fragmentation via Extension Headers), removes the hop-by-hop header checksum (relying on Layer 2 and Layer 4 checksums), and introduces a 20-bit Flow Label field for real-time QoS stream identification. NetMirage uses IPv4 private subnets (RFC 1918 10.0.0.0/8) to mirror standard industrial SCADA/enterprise network environments.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: Syllabus Module 3 (IP Version 6 - Addresses and Routing, Packet Format); implemented via standard IPv4 structures in app_build/backend/core/safety_policy.py (ipaddress.ip_network).

TOPIC FOCUS: MODULE 3: INTERNETWORKING

Q24: Multicast Addressing & Multicast Routing Protocols (DVMRP / IGMP)

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Multicast sends a single data stream simultaneously to a specific group of interested subscribers instead of blasting every device or sending duplicate individual packets. Protocols like IGMP let computers join groups, while DVMRP builds delivery trees to route packets efficiently.

DETAILED TECHNICAL BREAKDOWN:

IPv4 Multicast uses Class D addresses (224.0.0.0 to 239.255.255.255, leading bits 1110). Hosts join and leave multicast groups using Internet Group Management Protocol (IGMP) signaling over the local subnet. Multicast routing protocols construct delivery trees: Distance Vector Multicast Routing Protocol (DVMRP; RFC 1075) implements Reverse Path Forwarding (RPF) with flood-and-prune mechanics (truncating branches that have no downstream group members), whereas Protocol Independent Multicast (PIM) builds source-based or shared rendezvous-point distribution trees.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: Syllabus Module 3 (Multicast Addresses, Multicast Routing - DVMRP).

TOPIC FOCUS: MODULE 4: END TO END PROTOCOLS

Q25: UDP vs. TCP: Demultiplexer Architecture & Tradeoffs

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

UDP is a lightweight, connectionless protocol that sends datagrams immediately without handshakes or delivery guarantees, making it fast for DNS and streaming. TCP is a connection-oriented protocol that guarantees every byte arrives in order without errors, making it essential for web and SSH.

DETAILED TECHNICAL BREAKDOWN:

User Datagram Protocol (UDP; RFC 768) provides an ultra-minimal transport abstraction: an 8-byte header containing Source Port, Destination Port, Length, and Checksum. It acts as a simple port demultiplexer with no connection state, no flow control, and no congestion control. Transmission Control Protocol (TCP; RFC 793/9293) provides a reliable, full-duplex, ordered byte-stream abstraction over an unreliable network layer using a 20-byte base header, sequence/acknowledgment tracking, cumulative acknowledgments, sliding-window flow control, and adaptive congestion avoidance.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/intel/analyzer.py (L33-58 vs L93-110) categorizes TCP vs UDP packet behaviors.

TOPIC FOCUS: MODULE 4: END TO END PROTOCOLS

Q26: TCP 3-Way Handshake (SYN -> SYN-ACK -> ACK) Mechanics

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Before sending data, a client and server greet each other with three synchronizing packets: the client says 'Hello, let's sync' (SYN), the server replies 'I hear you, let's sync' (SYN-ACK), and the client confirms 'Got it, we are connected' (ACK). This handshake ensures both computers agree on sequence numbers and are ready to exchange data.

DETAILED TECHNICAL BREAKDOWN:

To establish a reliable bidirectional byte stream:

1. SYN: Client sends segment with flag SYN = 1, choosing a random Initial Sequence Number (ISN_c = x).

2. SYN-ACK: Server allocates TCB (Transmission Control Block) buffers, returns flag SYN = 1, ACK = 1, sets ACK_NUM = x + 1, and picks its own random Initial Sequence Number (ISN_s = y).

3. ACK: Client replies with flag ACK = 1, sets SEQ_NUM = x + 1 and ACK_NUM = y + 1. The connection enters the ESTABLISHED state.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/core/action_broker.py (ActionBroker._exec_connect_tcp, L119-124: logs 'TCP SYN handshake succeeded'), and app_build/backend/intel/analyzer.py (L43-58: records state as established with RTT).

TOPIC FOCUS: MODULE 4: END TO END PROTOCOLS

Q27: TCP Connection Termination (4-Way Handshake) & TIME_WAIT State

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Closing a TCP connection requires four steps because each direction of communication closes independently. After closing, the initiator waits in TIME_WAIT for twice the maximum packet lifetime to ensure any lost acknowledgments can be retransmitted and stray packets drain from the network.

DETAILED TECHNICAL BREAKDOWN:

TCP connection teardown is asymmetric:

1. Active closer sends FIN = 1 (SEQ = u). State: FIN_WAIT_1.

2. Passive closer returns ACK = 1 (ACK = u + 1). State: CLOSE_WAIT (active closer enters FIN_WAIT_2).

3. Passive closer finishes transmitting remaining buffered data and sends its own FIN = 1 (SEQ = v). State: LAST_ACK.

4. Active closer replies with ACK = 1 (ACK = v + 1) and transitions to TIME_WAIT.

The TIME_WAIT timer lasts for 2 * MSL (Maximum Segment Lifetime, typically 1 to 2 minutes). This ensures: (a) if the final ACK is lost, the passive closer's retransmitted FIN can be acknowledged; and (b) old duplicate segments from the closed incarnation expire and cannot corrupt future connections using the same socket 5-tuple.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: Syllabus Module 4 (Connection Establishment and Termination); observed in graceful session teardowns in app_build/backend/intel/honeypots.py.

TOPIC FOCUS: MODULE 4: END TO END PROTOCOLS

Q28: Half-Open TCP SYN Stealth Scans (T1046) & Port Discovery

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

A stealth SYN scan sends a SYN packet to see if a port is open, but when the target replies with SYN-ACK, the scanner immediately sends a reset (RST) packet instead of completing the handshake. This discovers open ports without opening a full connection, which used to evade basic application logs.

DETAILED TECHNICAL BREAKDOWN:

In a TCP Half-Open scan (popularized by nmap -sS), the scanner dispatches raw TCP segments with SYN = 1 to target ports. If a port is open and listening, the host kernel replies with SYN/ACK. Rather than sending the final ACK to complete the 3-way handshake, the scanner sends a RST (or ignores the packet), tearing down the half-open connection before the application layer's accept() system call is ever triggered. If the target port is closed, the host kernel returns a RST/ACK. NetMirage's protocol analyzer flags rapid multi-port SYN segments with incomplete handshakes as MITRE T1046 Network Service Discovery.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/intel/analyzer.py (L27-42: flags 'Sequential SYN sweep across registered lab segments, incomplete_handshakes: 3') and app_build/backend/intel/mitre_mapper.py (port_discovery -> T1046, 0.90 confidence, L20-25).

TOPIC FOCUS: MODULE 4: END TO END PROTOCOLS

Q29: TCP Segment Format & Control Flags

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

The TCP segment header is a 20-byte control label that includes source and destination ports, sequence numbers for ordering, and 6 one-bit flags that control connection setup, data pushes, resets, and teardowns.

DETAILED TECHNICAL BREAKDOWN:

Standard TCP header format (20 bytes base without options):
- Source Port (16 bits), Destination Port (16 bits)
- Sequence Number (32 bits), Acknowledgment Number (32 bits)
- Data Offset (4 bits: header length in 32-bit words), Reserved (3 bits)
- 6 Control Flags:
* URG: Urgent pointer field is valid
* ACK: Acknowledgment field is valid (mandatory on all packets after initial SYN)
* PSH: Push function (deliver buffered data to receiving application immediately)
* RST: Reset connection (abort upon error or closed port)
* SYN: Synchronize sequence numbers during connection establishment
* FIN: No more data from sender (close byte stream)
- Window Size (16 bits: flow control advertising available buffer space)
- Checksum (16 bits: covers header, payload, and IPv4 pseudo-header)
- Urgent Pointer (16 bits)

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/intel/analyzer.py (analyze_action_event and analyze_scapy_packet, L135-152) decodes TCP flags and header fields.

TOPIC FOCUS: MODULE 4: END TO END PROTOCOLS

Q30: Transport Layer Flow Control: Sliding Window & Receiver Window (rwnd)

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Flow control prevents a fast sender from flooding and crashing a slow receiver with more data than its memory can hold. The receiver constantly tells the sender how much free buffer space it has left using the rwnd window field in its acknowledgments.

DETAILED TECHNICAL BREAKDOWN:

TCP implements end-to-end flow control via a dynamic sliding byte window. The receiver advertises its available buffer space in the 16-bit Window Size field (rwnd) of every returned ACK. The sender is restricted by the constraint:

LastByteSent - LastByteAcked <= min(rwnd, cwnd)

If rwnd = 0, the sender must pause transmission. To prevent deadlock if an update advertising a reopened window is lost, the sender runs a Persist Timer: when it expires, the sender transmits a 1-byte Zero-Window Probe (ZWP), prompting the receiver to re-advertise its current rwnd.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: Syllabus Module 4 (Sliding Window & Flow Control).

TOPIC FOCUS: MODULE 4: END TO END PROTOCOLS

Q31: Adaptive Retransmission & RTT Estimation (Jacobson/Karels & Karn's Algorithm)

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Networks have varying speeds, so TCP continuously measures the round-trip time of packets to automatically calculate how long to wait before deciding a packet was lost. Jacobson's algorithm tracks both the average delay and its jitter, while Karn's algorithm prevents bad estimates by ignoring retransmitted packets.

DETAILED TECHNICAL BREAKDOWN:

TCP computes its Retransmission Timeout (RTO) dynamically using Jacobson and Karels' algorithm:

1. Measure sample round trip time RTT_s for an acknowledged segment.
2. Update Smoothed RTT: SRTT <- (1 - alpha) * SRTT + alpha * RTT_s (typically alpha = 0.125).
3. Update RTT Variation: RTTVAR <- (1 - beta) * RTTVAR + beta * |SRTT - RTT_s| (typically beta = 0.25).
4. Set Timeout: RTO = SRTT + 4 * RTTVAR.

Karn's Algorithm Rule: Never calculate RTT_s on segments that were retransmitted (because it is impossible to determine whether the ACK corresponds to the initial or retransmitted transmission, known as retransmission ambiguity). Upon timeout, invoke exponential backoff: RTO_new = 2 * RTO_old.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: Syllabus Module 4 & Lab Exp 7 (Adaptive Retransmission - Karn's, Jacobson/Karels). Evaluated in latency tracking in app_build/backend/core/action_broker.py (L50).

TOPIC FOCUS: MODULE 4: END TO END PROTOCOLS

Q32: Nagle's Algorithm vs. TCP_NODELAY in Interactive Terminal Sessions

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Nagle's algorithm bundles tiny keystroke packets together to save network overhead, but this introduces noticeable typing delay in terminal sessions. SSH honeypots disable Nagle's algorithm using TCP_NODELAY so every typed command is echoed back to the attacker immediately without lag.

DETAILED TECHNICAL BREAKDOWN:

Nagle's algorithm (RFC 896) solves the 'small-packet problem' (where sending 1 byte of user data prepended by 40 bytes of TCP/IP headers creates 97.5% protocol overhead). It dictates: if there is unacknowledged data in flight, buffer all outgoing user data until a full Maximum Segment Size (MSS) packet is accumulated or the pending ACK is received. However, in interactive SSH terminal sessions, Nagle interacts pathologically with Delayed ACKs (which wait up to 200 ms before returning an ACK), creating severe keystroke lag. NetMirage's Cowrie SSH honeypot sets socket.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1) to disable Nagle's algorithm, transmitting keystrokes immediately for authentic human shell interaction.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/intel/honeypots.py (Cowrie SSH session interaction, L53-87) and Syllabus Module 4 / Lab Exp 7.

TOPIC FOCUS: MODULE 4: END TO END PROTOCOLS

Q33: TCP Congestion Control: Slow Start, Congestion Avoidance, Fast Retransmit & Fast Recovery

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Congestion control protects the entire network from collapsing when too many devices transmit simultaneously. TCP starts by sending a few packets and doubles the rate every round trip (Slow Start) until it reaches a threshold, switches to linear growth, and cuts its speed when it detects dropped packets.

DETAILED TECHNICAL BREAKDOWN:

TCP congestion control (RFC 5681) regulates the Congestion Window (cwnd):

1. Slow Start: Begins with cwnd = 1 to 10 MSS. For every received ACK, cwnd <- cwnd + 1 MSS (exponential growth per RTT) until cwnd >= ssthresh (slow start threshold).

2. Congestion Avoidance: Once cwnd >= ssthresh, cwnd <- cwnd + (1 / cwnd) MSS per ACK (Additive Increase: +1 MSS per RTT).

3. Fast Retransmit: Upon receiving 3 duplicate ACKs (signaling a single lost packet while later packets arrived), TCP retransmits the missing segment immediately without waiting for the RTO timer to expire.

4. Fast Recovery: Sets ssthresh <- max(cwnd/2, 2 MSS) and cwnd <- ssthresh + 3 MSS, growing cwnd for each additional duplicate ACK until the missing segment is acknowledged, then resumes Congestion Avoidance (Multiplicative Decrease / AIMD).

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: Syllabus Module 4 (TCP Congestion Control; Slow Start, Congestion Avoidance).

TOPIC FOCUS: MODULE 4: END TO END PROTOCOLS

Q34: Queuing Disciplines (FIFO vs. Fair Queuing) & Quality of Service (QoS)

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

FIFO queues packets like a simple grocery line where whoever arrives first leaves first, allowing heavy downloaders to starve other users. Fair Queuing separates traffic into independent lines so every connection gets an equal share of the network bandwidth.

DETAILED TECHNICAL BREAKDOWN:

First-In First-Out (FIFO) processes packets in strict arrival sequence and drops arriving packets when buffers overflow (Drop-Tail queuing), leading to global synchronization (where all TCP senders back off simultaneously). Fair Queuing (FQ) and Weighted Fair Queuing (WFQ) classify incoming packets into distinct conversation flows, serving queues round-robin in proportion to allocated bandwidth weights. This ensures flow isolation, guarantees bounded latency for interactive traffic (e.g., SSH, VoIP), and prevents single aggressive UDP flows from starving compliant TCP streams.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: Syllabus Module 4 (Queuing Disciplines - FIFO, Fair Queuing, Quality of Service).

TOPIC FOCUS: MODULE 4: END TO END PROTOCOLS

Q35: Remote Procedure Call (RPC) vs. Action Broker Parametric Dispatch

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

RPC allows a computer program to call a function on another computer as if it were a local subroutine, handling all serialization in the background. NetMirage's Action Broker uses this same idea to turn high-level AI action requests into strongly-typed network calls without raw shell execution.

DETAILED TECHNICAL BREAKDOWN:

Remote Procedure Call (RPC) abstracts network communication behind standard programming function calls using client stubs, server stubs, and serialization/marshalling (such as JSON-RPC, gRPC/Protobuf). The client stub serializes procedure arguments into a network representation, transmits them over transport sockets, and deserializes returned responses. NetMirage's Action Broker implements this architectural design pattern: high-level AI requests (action_type: 'resolve_dns', parameters: {'query': '...'}) are marshalled into strongly-typed Python functions (_exec_resolve_dns) that execute isolated network probes without shell invocation.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/core/action_broker.py (ActionBroker.execute_action, L15-60) and Syllabus Module 4 / Lab Exp 8 (Remote Procedure Call).

TOPIC FOCUS: MODULE 5: END TO END DATA & SECURITY

Q36: IP Security (IPsec) Architecture: AH vs. ESP & Modes

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

IPsec adds cryptographic security directly into Layer 3 so all communication between two devices is authenticated and encrypted regardless of what application is running. It uses Authentication Header (AH) to verify who sent the packet and Encapsulating Security Payload (ESP) to encrypt the contents.

DETAILED TECHNICAL BREAKDOWN:

IPsec (RFC 4301) operates at the Network Layer using two primary protocols: Authentication Header (AH; IP protocol 51), which provides data origin authentication, data integrity, and anti-replay protection (but zero encryption, hashing the payload and immutable IP header fields); and Encapsulating Security Payload (ESP; IP protocol 50), which provides confidentiality via symmetric ciphers (e.g. AES-GCM) alongside integrity and authentication. IPsec operates in two modes: Transport Mode (encrypting only the transport payload while retaining the original IP header, used for host-to-host links); and Tunnel Mode (encapsulating the entire original IP datagram inside a new outer IP header, used for gateway-to-gateway VPN tunnels).

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: Syllabus Module 5 (IP Security - IPsec, Trust and Threats, Firewalls).

TOPIC FOCUS: MODULE 5: END TO END DATA & SECURITY

Q37: Packet Filtering vs. Stateful Inspection Firewalls

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

A packet filter inspects each packet in isolation like a bouncer checking individual tickets at a door without remembering who came inside. A stateful firewall tracks the complete ongoing conversation so return traffic for an established connection is allowed back automatically.

DETAILED TECHNICAL BREAKDOWN:

Stateless Packet Filters inspect each packet independently against an Access Control List (ACL) evaluating only static header fields: {Source IP, Destination IP, Protocol, Source Port, Destination Port}. They cannot verify whether a TCP segment represents an initial SYN or a valid ongoing session, making them vulnerable to ACK scans and spoofed flags. In contrast, Stateful Inspection Firewalls maintain a dynamic connection state table (such as Linux netfilter / conntrack): they track the TCP 3-way handshake, sequence number progressions, and connection state transitions (NEW, ESTABLISHED, RELATED, INVALID). Return traffic for established sessions is dynamically permitted without opening wide static port ranges.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: Syllabus Module 5 (Firewalls); implemented via NetMirage's Docker isolation filters and connection state tracking in app_build/backend/core/safety_policy.py.

TOPIC FOCUS: MODULE 5 & 6: CONTEMPORARY ISSUES

Q38: Wireshark Packet Analysis: Capture Filters (BPF) vs. Display Filters

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Capture filters tell the network card what packets to save before recording begins so your hard drive doesn't fill up with junk traffic. Display filters let you search and filter through already recorded packets on your screen without deleting anything.

DETAILED TECHNICAL BREAKDOWN:

Wireshark relies on libpcap/WinPcap for low-level packet capture. Capture Filters are defined using Berkeley Packet Filter (BPF) syntax (e.g., 'tcp port 80 and net 10.0.10.0/24') and are evaluated in the kernel before packets reach user space, drastically reducing packet loss during high-bandwidth sweeps. In contrast, Display Filters (e.g., 'ip.src == 10.0.10.15 and tcp.flags.syn == 1 and tcp.flags.ack == 0') operate post-capture in user memory: they evaluate the full protocol dissection tree, allowing complex string matches, HTTP URI lookups, and hex sequence queries across saved packet dumps.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: Syllabus Lab Exp 9 (Hands on Wireshark - Capture filter, Display filter, simple attacks). Emulated in real-time in app_build/frontend/src/components/AttackerTerminal.tsx and app_build/backend/intel/analyzer.py.

SECTION 2: PROJECT ARCHITECTURE & SECURITY DEFENSE QUESTIONS (16 QUESTIONS)

TOPIC FOCUS: ARCHITECTURE & SECURITY DEFENSE

Q39: The Non-Negotiable Boundary: Why Arbitrary Shell Commands Are Strictly Forbidden

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Many AI tools dangerously pass raw AI text directly into a command prompt, which allows an attacker or AI glitch to execute dangerous commands like deleting hard drives. NetMirage strictly bans shell execution, allowing the AI to request only 7 pre-coded, safe network actions with strict parameters.

DETAILED TECHNICAL BREAKDOWN:

NetMirage enforces a Zero-Arbitrary-Shell architectural invariant. Subsystem code strictly forbids subprocess.Popen(cmd, shell=True), os.system(), or passing LLM strings to command interpreters. The AI Planner can only select from a fixed enum of 7 allowlisted actions (discover_lab_services, resolve_dns, connect_tcp, request_http, attempt_ssh_to_lure, send_icmp_probe, inspect_session_output). Each action maps to parameterized Python networking functions that execute only bounded socket handshakes.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/core/action_broker.py (L6-60) and docs/SECURITY_BOUNDARIES.md (L56-78).

TOPIC FOCUS: ARCHITECTURE & SECURITY DEFENSE

Q40: Fail-Closed Security Policy: Behavior on Missing Configs, Out-of-CIDR Targets & Unmapped Ports

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Fail-closed means that if anything is unexpected, missing, or unverified, the system immediately says 'NO' and blocks the action rather than guessing or defaulting to allow. If a target IP or port is not on our strict approved list, the action is denied on the spot.

DETAILED TECHNICAL BREAKDOWN:

In security engineering, a fail-open system allows traffic when an error or missing configuration occurs, whereas a fail-closed system immediately denies all permissions. In safety_policy.py, any missing configuration, invalid run ID, target IP outside the 5 lab CIDRs (10.0.10.0/24 - 10.0.50.0/24), unregistered port (not in {22, 2222, 53, 80, 8080, 25, 587}), rate limit breach, or concurrency saturation instantly returns PolicyResult(allowed=False, status='denied').

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/core/safety_policy.py (SafetyPolicyEngine.evaluate_action, L91-175).

TOPIC FOCUS: ARCHITECTURE & SECURITY DEFENSE

Q41: Safety Policy Engine vs. Action Broker: Separation of Verification and Execution

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

The Safety Policy Engine is the security guard that checks badges and denies unauthorized requests, while the Action Broker is the technician that actually performs approved work. Keeping them separate guarantees that execution code never bypasses security checks.

DETAILED TECHNICAL BREAKDOWN:

NetMirage enforces separation of concerns between verification and execution. The Safety Policy Engine (safety_policy.py) is a pure authorization filter that evaluates proposed actions against mathematical invariants (CIDRs, ports, rates, concurrency, run status) without touching network interfaces. The Action Broker (action_broker.py) is an execution translator that dispatches only pre-approved actions across isolated sockets.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/orchestration/lab_runner.py (L128-163 demonstrates that action_broker.execute_action is only invoked if policy_eval.allowed is True).

TOPIC FOCUS: ARCHITECTURE & SECURITY DEFENSE

Q42: The AI Attacker 3-Layer Loop: Observation Summary -> Planner -> Safe Action Request

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

The attacker engine works like a human analyst: it first summarizes what it currently sees, decides its next logical move using a state machine, and outputs a structured request to test a specific IP and port.

DETAILED TECHNICAL BREAKDOWN:

The adversary emulation loop runs a 3-layer cycle:

1. Layer 1 (build_observation_summary): Aggregates prior executed actions, statuses, open ports, and discovered IPs into an ObservationSummary dataclass.

2. Layer 2 (plan_next_action): Evaluates current progression stage to select the next adversary tactic (reconnaissance, DNS discovery, web probing, credential brute force, lateral ICMP probe).

3. Layer 3 (SafeActionRequest): Emits a strongly-typed request containing action_type, target_ip, target_port, and parameters.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/core/planner.py (AIPlanner, L22-136).

TOPIC FOCUS: ARCHITECTURE & SECURITY DEFENSE

Q43: Passive Telemetry Collection (Scapy/Zeek) Without Host Reconfiguration

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Passive monitoring works like a wiretap that silently listens to network packets flowing by without changing firewall rules or alerting the attacker. This lets defenders gather intelligence without breaking running applications.

DETAILED TECHNICAL BREAKDOWN:

Active monitoring requires injecting agents or modifying host firewall (iptables) rules, which alerts sophisticated adversaries. Passive telemetry operates on mirrored packet streams (such as Docker bridge taps or promiscuous socket sniffing in the monitor subnet 10.0.50.0/24). Scapy and Zeek parse frame headers, track TCP state machines, calculate packet entropy, and detect port sweeps completely out-of-band without modifying live host routing or alerting the attacker.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/intel/analyzer.py (ProtocolAnalyzer, L5-154) and app_build/docker/telemetry-collector/.

TOPIC FOCUS: ARCHITECTURE & SECURITY DEFENSE

Q44: MITRE ATT&CK; Classification Engine & Confidence Score Math

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

The MITRE ATT&CK; mapper matches observed network behavior against a globally recognized encyclopedia of cyberattack techniques. It assigns a confidence score between 0.0 and 1.0 based on how strongly the evidence matches known attack patterns.

DETAILED TECHNICAL BREAKDOWN:

The mapper evaluates protocol observations against predefined behavioral rules:

- Multi-port TCP SYN sweep -> T1046: Network Service Discovery (Confidence: 0.90, Profile: reconnaissance)

- Web path traversal (/admin/login) -> T1190: Exploit Public-Facing Application (Confidence: 0.85, Profile: reconnaissance)

- Rapid SSH authentication -> T1110: Brute Force (Confidence: 0.95, Profile: credential_probing)

- SSH remote session execution -> T1021.004: Remote Services: SSH (Confidence: 0.92, Profile: credential_probing)

- Large or anomalous DNS query -> T1071.004: Application Layer Protocol: DNS (Confidence: 0.88, Profile: covert_channel)

Dynamic cumulative threat score: S_threat = min(100, S_threat + floor(confidence * 25)).

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/intel/mitre_mapper.py (MitreAttackMapper.MAPPING_RULES, L19-68) and app_build/backend/orchestration/lab_runner.py (L258-278).

TOPIC FOCUS: ARCHITECTURE & SECURITY DEFENSE

Q45: Deception Policy Engine & Dynamic Traffic Diversion

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Instead of simply blocking an attacker, the deception engine outsmarts them by dynamically steering their traffic into realistic fake servers. As the attacker becomes more aggressive, they are routed deeper into specialized honeypots or quarantined completely.

DETAILED TECHNICAL BREAKDOWN:

Upon receipt of classified MITRE detections, the Deception Policy Engine (deception_policy.py) evaluates the active threat profile and confidence:

- reconnaissance: Diverts attacker to interactive Cowrie SSH (10.0.20.10) or Web decoy (10.0.30.10) to absorb probing.

- credential_probing: Reroutes traffic deeper into simulated corporate core services (services-lure 10.0.40.0/24).
- covert_channel: Mirrors traffic to the DNS honeynet tap (10.0.40.10) to inspect exfiltration payloads.
- network_manipulation: Severs all active network paths and confines the attacker to the isolation quarantine sinkhole (10.0.50.99).

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/intel/deception_policy.py (DeceptionPolicyEngine.evaluate_policy, L12-52) and app_build/backend/orchestration/lab_runner.py (L280-300).

TOPIC FOCUS: ARCHITECTURE & SECURITY DEFENSE

Q46: Automated Credential Redaction ([REDACTED]) & Data Leakage Prevention

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

When attackers enter passwords or tokens during simulated attacks, NetMirage automatically scrubs them and replaces them with [REDACTED]. This prevents passwords from leaking into database logs, terminal screens, or live browser feeds.

DETAILED TECHNICAL BREAKDOWN:

Raw honeypot interactions often capture sensitive credentials, API keys, and environment variables. The redact_credentials utility recursively scans strings, lists, and dictionary keys matching sensitive tokens (password, secret, token, key, credential, auth_token, api_key). It applies regular expressions to sanitize key-value pairs (password=xyz), command line basic auth flags (-u user:pass), and HTTP Authorization: Basic headers, replacing values with [REDACTED] prior to SQLite insertion or WebSocket broadcast.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/intel/honeypots.py (redact_credentials, L7-46) and app_build/backend/routes.py (get_session & get_run_sessions, L118-154, L245-263).

TOPIC FOCUS: ARCHITECTURE & SECURITY DEFENSE

Q47: The Universal Event Envelope: Architecture, Schema & Inter-Subsystem Consistency

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

The Universal Event Envelope is a standard digital container used by every subsystem to package its messages. Because every component uses the exact same JSON format, the backend, database, and frontend communicate seamlessly without translation errors.

DETAILED TECHNICAL BREAKDOWN:

To eliminate schema drift and serialization friction across distributed microservices, all state changes conform to the strict Pydantic v2 contract:

{'run_id': str, 'event_id': str, 'timestamp': ISO8601_str, 'kind': str, 'source': str, 'payload': dict}

The kind discriminator (system, action, observation, detection, honeypot, policy) allows the EventBus and React frontend to route and render telemetry deterministically.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/schemas.py (EventEnvelope, L12-18), app_build/backend/event_bus.py (L183-194), and app_build/frontend/src/types.ts.

TOPIC FOCUS: ARCHITECTURE & SECURITY DEFENSE

Q48: Global Kill Switch (Terminate Button): Step-by-Step Takedown Mechanics

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Clicking the red Terminate button immediately kills the attack simulation in less than 50 milliseconds. It flips the database status to terminated, halts background tasks, and locks the dashboard screen so no further actions can run.

DETAILED TECHNICAL BREAKDOWN:

When the operator clicks TERMINATE on TopControlBar.tsx:

1. REST client sends POST /api/runs/{run_id}/terminate.
2. FastAPI updates models.Run.status = 'terminated' and records ended_at = utcnow().
3. lab_runner.terminate_run(run_id) cancels the running asyncio.Task (task.cancel()).
4. The loop catches asyncio.CancelledError, releases concurrency locks, and publishes a kind: 'system' envelope with status: 'terminated'.
5. The Safety Policy Engine immediately denies any subsequent queued actions (Global Kill Switch active).
6. The React console renders the locked TERMINATED state and disables run switches.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/orchestration/lab_runner.py (terminate_run, L57-71 & _run_loop, L321-336), app_build/backend/routes.py (L217-228), and app_build/frontend/src/components/TopControlBar.tsx (L107-123).

TOPIC FOCUS: ARCHITECTURE & SECURITY DEFENSE

Q49: Container Hardening & Jailbreak Prevention: cap_drop ALL, no-new-privileges, Read-Only Root

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Even if an attacker somehow breaks out of an emulated honeypot application, Docker locks them inside a digital straitjacket. The container has all root powers stripped, cannot gain new privileges, and cannot modify its own operating system files.

DETAILED TECHNICAL BREAKDOWN:

Every honeypot and gateway container in docker-compose.lab.yml enforces defence-in-depth kernel containment:

- security_opt: ['no-new-privileges:true']: Prevents setuid binaries (like sudo or su) from elevating privileges.
- cap_drop: [ALL]: Strips all 38 default Linux kernel capabilities (including CAP_NET_RAW, CAP_SYS_ADMIN, CAP_DAC_OVERRIDE).
- read_only: true: Mounts the root filesystem as read-only; attackers cannot overwrite system binaries or install rootkits.
- tmpfs: [/tmp:rw,noexec,nosuid]: Provides a temporary memory scratchpad with noexec and nosuid flags, blocking execution of downloaded binaries.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/docker/docker-compose.lab.yml (L55-61, L76-82, L97-103, L118-124, L139-145) and app_build/backend/core/safety_policy.py (validate_container_security, L83-90).

TOPIC FOCUS: ARCHITECTURE & SECURITY DEFENSE

Q50: Rate Limiting & Concurrency Guardrails in Cyber-Range Operations

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Rate limiting prevents the cyber-range from crashing or overheating by ensuring the AI attacker cannot spam hundreds of requests per second. NetMirage caps execution to 30 actions per minute and at most 2 actions running at the exact same moment.

DETAILED TECHNICAL BREAKDOWN:

High-frequency scanning can exhaust host socket buffers, trigger kernel out-of-memory (OOM) killers, or destabilize container bridges. The Safety Policy Engine tracks execution history in a sliding 60-second window (_action_timestamps[run_id]). If recent operations exceed MAX_ACTIONS_PER_MINUTE (default: 30), further requests are denied. Concurrency is governed by an atomic counter (_active_concurrency[run_id]); if active operations reach MAX_CONCURRENT_ACTIONS (default: 2), incoming actions are rejected until in-flight socket operations release their lock.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/core/safety_policy.py (check_rate_limit, L54-64 & acquire_concurrency, L71-82).

TOPIC FOCUS: ARCHITECTURE & SECURITY DEFENSE

Q51: WebSocket State Replay & Resilient Real-Time Event Streaming

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

If an operator reloads their browser or joins late, they don't miss anything that happened earlier in the test. The server stores all past events and instantly replays them to the browser the second it connects.

DETAILED TECHNICAL BREAKDOWN:

When a browser client opens a WebSocket connection to /api/runs/{run_id}/events, the backend calls event_bus.subscribe(run_id). The bus registers an asyncio.Queue for live event fanout and immediately iterates through _history[run_id], pushing all historical envelopes (kind: system, action, observation, detection, policy, honeypot) into the new queue before live streaming begins. This guarantees idempotent, complete state reconstruction on client reconnects.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: app_build/backend/event_bus.py (EventBus.subscribe, L18-30 & publish, L39-55) and app_build/backend/routes.py (L264-300).

TOPIC FOCUS: ARCHITECTURE & SECURITY DEFENSE

Q52: Database Entity Architecture & 6 Immutable SQLAlchemy Models

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

NetMirage saves all run data into 6 dedicated database tables so every simulated attack, defense reaction, and captured password has a permanent audit trail. We use an append-only design so historical forensic evidence can never be tampered with.

DETAILED TECHNICAL BREAKDOWN:

Designed using SQLAlchemy 2.0 with async/sync SQLite/PostgreSQL compatibility, the schema enforces 6 relational models:

1. Run: Top-level execution state, threat score, active profile, next deception zone, timestamps.
2. AgentAction: Audit log of proposed/executed actions, target IP/port, status (allowed/denied), latency, output.
3. Observation: Passive telemetry events, source/destination IPs, protocol, behavior type, raw payloads.
4. Detection: MITRE ATT&CK; technique classifications, confidence ratings, and behavioral evidence.
5. HoneypotSession: Captured terminal interactions, sanitized commands, redacted credentials, and banners.
6. PolicyDecision: Safety policy and deception routing verdicts, threat profiles, and target redirect zones.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: `app_build/backend/models.py (L18-112) and app_build/backend/event_bus.py (_persist_event, L56-182).`

TOPIC FOCUS: ARCHITECTURE & SECURITY DEFENSE

Q53: Front-End Telemetry Engineering: Monospace Typography, Tabular Nums & Zero Layout Jitter

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

Standard web fonts cause numbers to jump and vibrate when live metrics change rapidly on screen. NetMirage uses fixed-width monospace typography and tabular numbers so all tables, IP addresses, and timers remain rock-solid and easy to read.

DETAILED TECHNICAL BREAKDOWN:

In high-throughput cyber-range consoles, variable-width proportional fonts cause severe layout thrashing (jitter) as characters of varying pixel widths update in real-time. NetMirage locks the user interface to monospace typography (JetBrains Mono, IBM Plex Mono) with CSS tabular numbers (font-feature-settings: 'tnum' 1). The entire interface is structured as an invariant 100vh split-pane grid with zero body scroll, 1px high-contrast Charcoal borders (#111827), Ivory background (#FAF5E9), and zero decorative gradients or animations.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: `app_build/frontend/src/index.css (L1-35) and app_build/frontend/src/App.tsx.`

TOPIC FOCUS: ARCHITECTURE & SECURITY DEFENSE

Q54: Active Interaction Honeypots vs. Passive Network Taps in Cyber Defense

SIMPLE EXPLANATION (IN PLAIN ENGLISH):

A passive tap is a hidden camera that silently records network traffic without interacting, while an active honeypot is an undercover agent that talks back to the intruder to keep them busy. NetMirage uses both: passive taps detect the attack without alerting the adversary, and active honeypots absorb the attack to protect real servers.

DETAILED TECHNICAL BREAKDOWN:

Passive network taps (such as Scapy packet sniffers in monitor_net) mirror traffic out-of-band: they have zero IP addresses on monitored segments, generate zero packets, and can never be detected by adversary port scanners. In contrast, active honeypots (like Cowrie SSH on 10.0.20.10 or the Web decoy on 10.0.30.10) participate directly in network handshakes, allocating state, returning service banners, and accepting credentials to maximize adversary dwell time and extract high-fidelity threat intelligence. NetMirage combines both into a closed-loop defense pipeline.

WHERE IT IS IN NETMIRAGE (CODE & FUNCTION REFERENCE):

Path & Logic: `app_build/backend/intel/analyzer.py (passive tap) vs app_build/backend/intel/honeypots.py (active lure).`

SECTION 3: END-TO-END EXECUTION FLOW WALKTHROUGH (18 STEPS)

Architectural Lifecycle: The following 18-step sequential sequence traces an entire adversarial emulation run from the instant the operator initiates testing on the physical dashboard to final containment:

STEP 1: OPERATOR TRIGGERS RUN INITIALIZATION	**Component:** `TopControlBar.tsx`
Trigger / Action: Operator clicks tactile START RUN button on the hardware command strip.	**Operation:** Frontend issues asynchronous HTTP POST /api/runs with scenario payload {scenario: 'full_adversary_emulation'}. The switch enters active state with an instant offset depression animation.

STEP 2: RUN RECORD CREATED IN EVENT STORE	**Component:** `routes.py -> models.py`
Trigger / Action: FastAPI router receives request and commits initial Run entity.	**Operation:** Generates a deterministic run ID (e.g. run_20261018_1630_a1b2), initializes status='pending', threat_score=0, next_deception_zone='edge', and commits to SQLite/PostgreSQL.

STEP 3: ASYNCHRONOUS LAB RUNNER TASK SPAWNED	**Component:** `lab_runner.py`
Trigger / Action: LabRunner.start_run() initiates background orchestration worker.	**Operation:** Spawns an asyncio.Task running _run_loop(run_id). Updates run status to 'running' and sets started_at timestamp in the database.

STEP 4: SYSTEM RUN EVENT BROADCAST ON EVENTBUS	**Component:** `event_bus.py`
Trigger / Action: EventBus emits kind: 'system' status envelope.	**Operation:** Envelope {run_id, kind: 'system', source: 'system', payload: {status: 'running'}} is published to subscribers and committed to the immutable event log.

STEP 5: WEBSOCKET STREAMS INITIAL STATE TO DASHBOARD	**Component:** `websocket.ts -> App.tsx`
Trigger / Action: Browser WebSocket client receives initial state frame.	**Operation:** TopControlBar indicator flips from STANDBY to LIVE [#]. The timer starts advancing with tabular monospace digits.

STEP 6: LAYER 1: AI PLANNER AGGREGATES OBSERVATIONS	**Component:** `planner.py (AIPlanner)`
Trigger / Action: AIPlanner.build_observation_summary() compiles historical intelligence.	**Operation:** Constructs ObservationSummary dataclass aggregating vantage point (10.0.10.15), discovered IP nodes, open ports, and prior action statuses.

STEP 7: LAYER 2: AI PLANNER FORMULATES SAFE ACTION REQUEST	**Component:** `planner.py (AIPlanner)`
Trigger / Action: AIPlanner.plan_next_action() selects next progression step.	**Operation:** Selects action (e.g. Stage 0: discover_lab_services on 10.0.10.42:80; or Stage 3: attempt_ssh_to_lure on 10.0.20.10:22) and outputs SafeActionRequest.

STEP 8: LAYER 3: SAFETY POLICY ENGINE EVALUATES GUARDRAILS	**Component:** `safety_policy.py`
Trigger / Action: SafetyPolicyEngine.evaluate_action() executes fail-closed inspection.	**Operation:** Validates: (1) action in 7 allowlisted types, (2) target IP in allowed CIDRs (10.0.10.0/24 - 10.0.50.0/24), (3) target port in allowed ports, (4) rate limit < 30/min, (5) concurrency < 2, (6) kill switch inactive. Returns PolicyResult(allowed=True).

STEP 9: ACTION BROKER EXECUTES PARAMETRIC SOCKET OPERATION	**Component:** `action_broker.py`
Trigger / Action: ActionBroker.execute_action() executes isolated network probe.	**Operation:** Calls dedicated handler (e.g. _exec_connect_tcp, _exec_attempt_ssh_to_lure) using socket/asyncio. Monotonic timer (time.perf_counter()) records exact latency_ms. No arbitrary shell is invoked.

STEP 10: ACTION RESULT PERSISTED AND BROADCAST	**Component:** `event_bus.py -> AttackerTerminal.tsx`

Trigger / Action: Action execution envelope logged to DB and pushed via WebSocket.	**Operation:** EventBus publishes kind: 'action' envelope. Panel [02] (AttackerTerminal) appends a new Wireshark-density line: timestamp, stage index, action name, target IP:port, ALLOWED status, and latency_ms.

STEP 11: HONEYPOT INTERCEPTS INTERACTION & REDACTS CREDENTIALS	**Component:** `honeypots.py (HoneypotNormalizer)`
Trigger / Action: Honeypot adapter sanitizes captured credentials and logs session.	**Operation:** If action targeted a lure, HoneypotNormalizer sanitizes passwords/tokens via redact_credentials() to [REDACTED]. Emits kind: 'honeypot' envelope. Stored in honeypot_sessions table.

STEP 12: PASSIVE PROTOCOL ANALYZER DERIVES BEHAVIORAL TELEMETRY	**Component:** `analyzer.py (ProtocolAnalyzer)`
Trigger / Action: ProtocolAnalyzer.analyze_action_event() extracts packet observations.	**Operation:** Examines protocol headers and behavioral patterns out-of-band (e.g. port_discovery, tcp_syn_pattern, ssh_credential_probing). Emits kind: 'observation' envelope.

STEP 13: MITRE ATT&CK; CLASSIFIER EVALUATES OBSERVATIONS	**Component:** `mitre_mapper.py`
Trigger / Action: MitreAttackMapper.classify_observation() matches technique rules.	**Operation:** Correlates behavior into MITRE technique (e.g. T1046 Network Service Discovery, T1110 Brute Force), assigns confidence score (0.80 to 0.95), and determines active threat profile.

STEP 14: THREAT SCORE UPDATED & DETECTION BROADCAST	**Component:** `lab_runner.py -> DetectionsMatrix.tsx`
Trigger / Action: Cumulative threat index incremented and Detection emitted.	**Operation:** Threat index updated: S_threat = min(100, S_threat + floor(confidence * 25)). Emits kind: 'detection' envelope. Panel [03] (DetectionsMatrix) renders technique badge and 10-segment meter (e.g. [[##########] 0.86]).

STEP 15: DECEPTION POLICY ROUTER DETERMINES TRAFFIC VECTOR	**Component:** `deception_policy.py`
Trigger / Action: DeceptionPolicyEngine.evaluate_policy() calculates dynamic reroute.	**Operation:** Based on threat profile (reconnaissance -> ssh-lure/web-lure; credential_probing -> services-lure; network_manipulation -> quarantine), generates routing decision and sets next_deception_zone.

STEP 16: POLICY DECISION PERSISTED & SCHEMATIC CANVAS UPDATED	**Component:** `event_bus.py -> NetworkTopology.tsx`
Trigger / Action: Routing update emitted and rendered on SVG topology map.	**Operation:** Emits kind: 'policy' envelope. Panel [01] (NetworkTopology) updates highlighted vector lines, illuminating the active reroute path (e.g. 10.0.10.15 -> 10.0.20.10) in Tangerine Yellow (#FFCC00).

STEP 17: HONEYPOT INSPECTOR POPULATES DECOY SESSION FEEDS	**Component:** `HoneypotInspector.tsx`
Trigger / Action: Captured terminal commands and sanitized payloads display in UI.	**Operation:** Panel [04] (HoneypotInspector) updates session count badge. Clicking a session opens the split payload drawer displaying banners, captured commands, and redacted credentials.

STEP 18: SCENARIO COMPLETION OR GLOBAL KILL SWITCH TAKEDOWN	**Component:** `lab_runner.py -> TopControlBar.tsx`
Trigger / Action: Emulation completes all stages or operator hits TERMINATE.	**Operation:** Upon final stage completion, status transitions to 'completed'. If operator clicks TERMINATE at any point, kill switch cancels task within 50ms, releases locks, and locks UI in red TERMINATED state.

SECTION 4: QUICK CODE-NAVIGATION CHEAT SHEET TABLE

Candidate Defense Matrix: Quick-reference index mapping each candidate to their core subsystem responsibilities, exact files, function names, and screen talking points during examiner cross-questioning.

Candidate	Topic Examiner Asks About	Exact File Path to Open	Function / Class	What to Point Out on Screen
Abhisumat Panda (25BYB0202)	**Zero-Egress Docker Isolation (internal: true)**	`app_build/docker/docker-compose.lab.yml`	`networks: {edge_net, ssh_net, web_net, services_net, monitor_net}`	Show 'internal: true' on lines 6, 15, 24, 33, 42. Explain that Linux kernel iptables drops all packets routed outside the bridge, preventing internet breakout.
Abhisumat Panda (25BYB0202)	**Container Hardening (cap_drop ALL & read_only)**	`app_build/docker/docker-compose.lab.yml`	`cap_drop: [ALL], security_opt, tmpfs`	Show cap_drop ALL stripping root powers (L58, L79, L100), read_only: true root filesystems, and tmpfs memory scratchpads blocking binary execution.
Abhisumat Panda (25BYB0202)	**5-VLAN Lab Network Architecture Math**	`app_build/docker/docker-compose.lab.yml`	`ipam: config: subnet: 10.0.x.0/24`	Explain 5 /24 CIDRs (10.0.10.0/24 to 10.0.50.0/24), 254 usable host addresses, gateway .1, and host allocations on the chalkboard/screen.
Atharva Parmar (25BYB0180)	**Passive Protocol Telemetry & Scapy Engine**	`app_build/backend/intel/analyzer.py`	`ProtocolAnalyzer.analyze_scapy_packet() & analyze_action_event()`	Show Ether/IP/TCP decapsulation via Scapy (L130-153). Explain out-of-band packet sniffing without modifying live host iptables firewall rules.
Atharva Parmar (25BYB0180)	**Anomaly Detection (SYN Sweeps, DNS Tunneling, ARP)**	`app_build/backend/intel/analyzer.py`	`ProtocolAnalyzer behavioral classifiers`	Show TCP SYN sweeps (L27-42), HTTP path probing (L59-75), DNS exfil keyword checks (L93-110), and ICMP payload entropy metrics.
Atharva Parmar (25BYB0180)	**MITRE ATT&CK; Mapping & Confidence Scoring**	`app_build/backend/intel/mitre_mapper.py`	`MitreAttackMapper.MAPPING_RULES & classify_observation()`	Show rule dictionary (L19-68): port_discovery -> T1046 (0.90), web_path_probing -> T1190 (0.85), ssh_credential_probing -> T1110 (0.95), and threat scoring.
Manas Patel (25BYB0190)	**Cowrie SSH, Web, DNS & SMTP Lure Adapters**	`app_build/backend/intel/honeypots.py`	`HoneypotNormalizer: normalize_cowrie_event, normalize_web_event`	Show event normalizers for SSH (L54-87), Web (L89-118), DNS (L120-145), and SMTP (L147-172). Point out port bindings and banner emulation.
Manas Patel (25BYB0190)	**Automated Credential Redaction Engine**	`app_build/backend/intel/honeypots.py`	`redact_credentials()`	Show recursive dictionary cleaner and regex sanitizers (L9-45). Point out that passwords, secret keys, and -u user:pass CLI flags become [REDACTED].
Manas Patel (25BYB0190)	**Dynamic Deception Routing & Threat Profiles**	`app_build/backend/intel/deception_policy.py`	`DeceptionPolicyEngine.evaluate_policy()`	Show dynamic diversion branches (L24-42): reconnaissance -> ssh-lure/web-lure; credential_probing -> services-lure; network_manipulation -> quarantine sinkhole.
Varun Agarwal (25BYB0186)	**7-Action Allowlist & Anti-Shell Invariant**	`app_build/backend/core/safety_policy.py`	`ALLOWLISTED_ACTIONS & SafetyPolicyEngine.evaluate_action()`	Point out the 7 strings in ALLOWLISTED_ACTIONS (L10-18). Show that any action not in this set immediately returns status: 'denied'. Emphasize no shell=True exists.
Varun Agarwal (25BYB0186)	**Fail-Closed CIDR Boundary & Port Filtering**	`app_build/backend/core/safety_policy.py`	`is_ip_in_lab() & is_port_allowed()`	Show IP matching with ipaddress.ip_network (L44-49). If an IP is outside 10.0.10-50.0/24 or port not in {22, 53, 80, 8080, 25, 587}, it is blocked immediately.

Varun Agarwal (25BYB0186)	**Rate Limiting & Concurrency Guardrails**	`app_build/backend/core/safety_policy.py`	`check_rate_limit()` & `acquire_concurrency()`	Show 60-second sliding window timestamps (L54-64) enforcing max 30 actions/min, and atomic counter (L71-82) capping concurrent operations at 2.
Varun Agarwal (25BYB0186)	**Parametric Socket Dispatch & Latency Timing**	`app_build/backend/core/action_broker.py`	`ActionBroker.execute_action()` & `_exec_connect_tcp()`	Show that execute_action() uses time.perf_counter() (L24, L50) to measure latency_ms. Show that each action calls a pure Python socket method without spawning bash.
Varun Agarwal (25BYB0186)	**Universal Event Envelope & Pub/Sub Fan-out**	`app_build/backend/event_bus.py`	`EventBus.subscribe()`, `publish()`, & `emit()`	Show that emit() creates a uniform JSON envelope (L183-194). Show historical replay in subscribe() (L25-29) so newly connected WebSockets get past events.
Varun Agarwal (25BYB0186)	**Global Kill Switch (Terminate Button Mechanics)**	`app_build/backend/orchestration/lab_runner.py`	`LabRunner.terminate_run()` & `_run_loop cancellation`	Show task.cancel() (L68-70) and asyncio.CancelledError handler (L321-336). Show database status updated to 'terminated' within 50 milliseconds.
Varun Agarwal (25BYB0186)	**Hardware Console UI & Monospace Tabular Nums**	`app_build/frontend/src/index.css & App.tsx`	`Tailwind configuration, CSS tokens, 100vh grid layout`	Show font-feature-settings: 'tnum' 1 in index.css (L10-15) preventing layout jitter. Point out strict color tokens: Ivory #FAF5E9, Green #009B4D, Tangerine #FFCC00.